数据结构

关于成绩

平时成绩：30%

（乐学作业、课堂出勤、上机实训）

期末考试成绩：70%

目的

通过本课程的学习，运用本课程讨论的知识更好地进行算法设计与算法分析，掌握计算机进行数据处理的基本原理、基本方法和技巧，进一步提高程序设计的水平和能力。

本课程的特点：

1 计算机专业重要的专业基础课之一.

2 需要有关“程序设计语言”和“离散数学”的知识作为课程的基础.

3. 实践性较强.

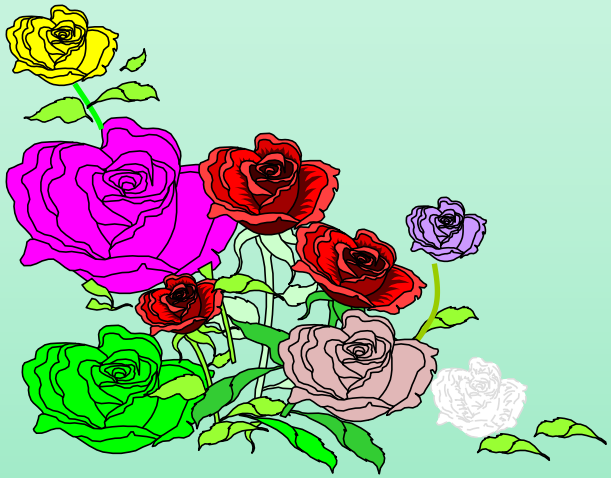
4. 难学而且重要.

如何学好数据结构

- 1、尽可能的认真独立完成作业。
- 2、多上机练习(多动手、多思考)

祝同学们：

学习新课程愉快！



学习进步！

第一章

绪论

主要内容

1.1 什么是数据结构

1.2 基本概念和术语

1.3 抽象数据类型的表示与实现

1.4 算法和算法分析

1.1 什么是数据结构

算法 + 数据结构 = 程序

程序设计：为计算机处理问题编制一组指令集

算法（怎么样处理）：处理问题的策略

数据结构（被处理的信息如何表示）：问题的数学模型

数据处理

- 处理数据的种类和能力

  数（整数、实数）

  字符、文字、图形、图像、声音

计算机应用

数值　　非数值

- 为了编写“好”的程序，必须分析待处理对象的特性以及各处理对象之间存在的关系

用计算机解决一具体问题时，大致需要经过下列几个步骤：

- 1、从具体问题抽象出数学模型；
- 2、设计解此数学模型的算法；
- 3、编写程序、测试、调整直至得到最终解答；

如何寻求数据模型？

- 1、分析问题；
- 2、提取操作的对象；
- 3、找出操作对象之间的关系；
- 4、用数学的语言加以描述；

例如：数值计算的程序设计问题

结构静力分析计算

—— 线性代数方程组

全球天气预报

—— 环流模式方程
（球面坐标系）

非数值计算的程序设计问题

例一：求一组(n个)整数中的最大值

算法：　基本操作是“比较两个数的大小”

模型：　取决于整数值的范围

例二：计算机对弈

算法：？ 对弈的规则和策略

模型：？ 棋盘及棋盘的格局

例三：足协的数据库管理

算法：　　需要管理的项目？

如何管理？　用户界面？

模型：　　各种表格

概括地说：

数据结构是一门讨论“描述现实世界实体的数学模型(非数值计算)及其上的操作在计算机中如何表示和实现”的学科。

具体的说：

数据结构主要研究数据之间有哪些结构关系，如何表示，如何存储，如何处理

1.2 基本概念和术语

- 一、数据与数据结构
- 二、数据类型
- 三、抽象数据类型

一、数据与数据结构

数据：

描述客观事物的数字、字符以及一切能够输入到计算机中，并且能够被计算机程序处理的符号的集合。

- 是计算机操作的对象的总称.
- 是计算机处理的信息的某种特定的符号表示形式

数据元素：

是数据（集合）中的一个“个体”

是数据结构中讨论的基本单位

数据项：是数据结构中讨论的最小单位

数据元素可以是数据项的集合

例如：描述一个运动员的数据元素可以是

姓名	俱乐部名称	出生日期	参加日期	职务	业绩

年	月	日

称之为组合项

数据结构：

1、数据元素之间的联系称之为结构

2、数据结构就是具有结构的数据元素的集合

例如 用三个 4 位的十进制数表示一个含 12 位数的十进制数

例如:

3214,6587,9345 — **a1**(3214),**a2**(6587),**a3**(9345)

则在数据元素 a1、a2 和 a3 之间存在着

“次序”关系 **<a1,a2>、<a2,a3>**

3214，6587，9345 ≠ 6587，3214，9345

a1 a2 a3 a2 a1 a3

又例，在2行3列的二维数组{a1, a2, a3, a4, a5, a6}中六个元素之间存在两个关系:

a1	a2	a3
a4	a5	a6

行的次序关系:

row = {<a1,a2>,<a2,a3>,<a4,a5>,<a5,a6>}

列的次序关系:

col = {<a1,a4>,<a2,a5>,<a3,a6>}

a1	a3	a5
a2	a4	a6

$\neq$

a1	a2	a3
a4	a5	a6

再例，在一维数组 {a1, a2, a3, a4, a5, a6} 的数据元素之间存在如下的**次序关系**:

$$\{<a_i, a_{i+1}>| \ i=1, 2, 3, 4, 5\}$$

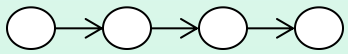
可见，不同的“关系”构成不同的“结构”或者说，数据结构是相互之间存在着某种逻辑关系的数据元素的集合。

逻辑结构和物理结构(存储结构)

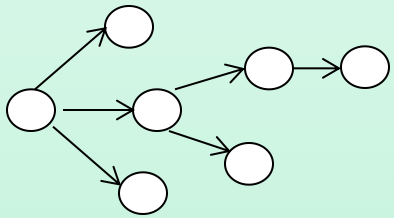
- 数据结构定义中的关系描述的是数据元素之间的逻辑关系，因此，又称为数据的逻辑结构。
- 数据结构在计算机中的表示称为数据的物理结构，又称为存储结构，它包括数据元素的表示和关系的表示。

数据的**逻辑结构**可归结为以下**四类**:

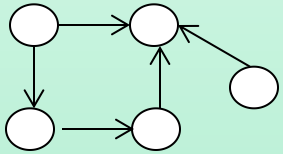
线性结构



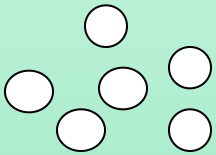
树形结构



图状结构



集合结构



数据结构的形式定义为:

数据结构是一个二元组

$$Data_Structures = (D, S)$$

其中:D 是数据元素的有限集,

S 是 D上关系的有限集。

关系的映象方法（表示<x, y>的方法）

数据元素之间的关系在计算机中有两种不同的表示方法:顺序映象和非顺序映象。

1 顺序映象

- 对应的存储结构：顺序存储结构。
- 顺序映象的特点是借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系。

2 非顺序映象

- 对应的存储结构：链式存储结构。
- 非顺序映象的特点是借助指示元素存储地址的指针表示数据元素之间的逻辑关系。

顺序映象

以相对的存储位置表示后继关系

例如:令 y 的存储位置和 x 的存储位置之间差一个常量 C

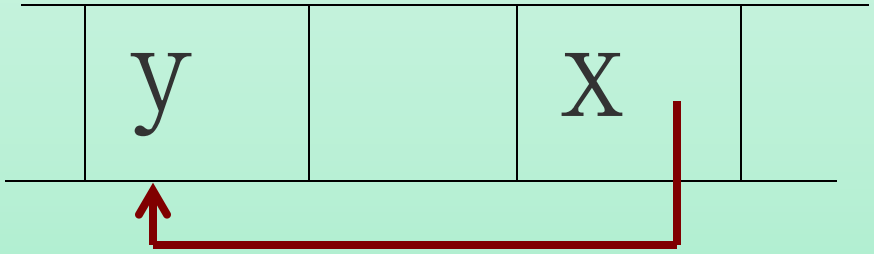
而 C 是一个隐含值，**整个存储结构中只含数据元素本身的信息**

	x		y

非顺序映象（链式映象）

以附加信息(指针)表示后继关系

需要用一个和 x 在一起的附加信息指示 y 的存储位置



数据的逻辑结构与存储结构的关系

- 任何一个

算法的设计取决于选定的数据（逻辑）结构

算法的实现依赖于采用的存储结构

在不同的编程环境中，存储结构可有不同的描述方法。

当用高级程序设计语言进行编程时，通常可用高级编程语言中提供的数据类型描述之。

例如:

以三个带有次序关系的整数表示一个长整数时，可利用 **C** 语言中提供的整数数组类型。

定义长整数为:

typedef int Long_int [3]

数据结构研究的主要内容

- 逻辑结构　研究数据之间固有的客观联系
- 存储结构　研究数据在计算机内部的存储方法
- 算法　研究如何在数据的各种结构（逻辑的和物理的）上实施有效的操作或处理

二、数据类型

在用高级程序语言编写的程序中，必须对程序中出现的每个变量、常量或表达式，明确说明它们所属的数据类型。

例如，C 语言中提供的基本数据类型有:

整型 int

浮点型 float

双精度型 double

} 实型

字符型 char

逻辑型 bool （C++语言）

C语言中提供的复合数据类型有：

数组：　[]

结构：　struct

不同类型的变量，其所能取的值的范围不同，所能进行的操作不同。

数据类型 是一个 值的集合和定义在此集合上的 一组操作的总称。

一个值的集合 + 定义在此集合上的一组操作 = 数据类型

1.3 抽象数据类型

（Abstract Data Type 简称ADT）

是指一个数学模型以及定义在此数学模型上的一组操作。

数据结构 + 定义在此数据结构上的一组操作 = 抽象数据类型

- 抽象数据类型的定义仅取决于它的一组逻辑特性，而与其在计算机内部如何表示和实现无关，即只要其数学特性不变，不影响外部使用
- 抽象数据类型和数据类型实际上是一个概念

例如，抽象数据类型复数的定义：

ADT Complex {

数据对象：

D＝{e1,e2 ｜ e1,e2∈RealSet }

数据关系：

R1＝{<e1,e2> | e1是复数的实数部分

| e2 是复数的虚数部分 }

基本操作：

AssignComplex(&Z, v1, v2)

操作结果：构造复数 Z, 其实部和虚部分别被赋以参数 v1 和 v2 的值。

DestroyComplex(&Z)

操作结果：复数Z被销毁。

GetReal(Z, &realPart)

初始条件：复数已存在。

操作结果：用**realPart**返回复数Z的实部值。

GetImag(Z, &ImagPart)

初始条件：复数已存在。

操作结果：用ImagPart返回复数Z的虚部值

Add(z1,z2, &sum)

初始条件：z1, z2是复数。

操作结果：用sum返回两个复数z1, z2 的和值。

} ADT Complex

假设:z1和z2是上述定义的复数

则

Add(z1, z2, z3)

操作的结果即为用户所求的结果

z3 = z1 + z2

抽象数据类型的两个特征：

数据抽象 用ADT描述程序处理的实体时，强调的是其本质的特征、其所能完成的功能以及它和外部用户的接口（即外界使用它的方法）。

数据封装 将实体的外部特性和其内部实现细节分离，并且对外部用户隐藏其内部实现细节。

抽象数据类型的描述方法

抽象数据类型可用（D，S，P）三元组表示

其中：D 是数据对象；

S 是 D 上的关系集；

P 是对 D 的基本操作集。

ADT 抽象数据类型名 {

数据对象：〈数据对象的定义〉

数据关系：〈数据关系的定义〉

基本操作：〈基本操作的定义〉

} ADT 抽象数据类型名

其中基本操作的定义格式为:

基本操作名（参数表）

初始条件：〈初始条件描述〉

操作结果：〈操作结果描述〉

赋值参数 只为操作提供输入值。

引用参数 以&打头，除可提供输入值外，还将返回操作结果。

初始条件 描述了操作执行之前数据结构和参数应满足的条件，若不满足，则操作失败，并返回相应出错信息。

操作结果 说明了操作正常完成之后，数据结构的变化状况和应返回的结果。若初始条件为空，则省略之。

1.3 抽象数据类型的表示和实现

抽象数据类型需要通过固有数据类型(高级编程语言中已实现的数据类型)来实现。即利用处理器中已经操作的数据类型来说明新的结构，用已经实现的操作来组合新的操作。

例如，对以上定义的复数

```
// -----存储结构的定义
typedef  struct {
    float realpart;
    float imagpart;
}complex;

// -----基本操作的函数原型说明

void Assign( complex &Z,
                    float realval, float imagval );
//构造复数 Z, 其实部和虚部分别被赋以参数
// realval 和 imagval 的值
```

```
float GetReal( cpmplex Z );
  // 返回复数 Z 的实部值

float Getimag( cpmplex Z );
  // 返回复数 Z 的虚部值

void add( complex z1, complex z2,
                                complex &sum );
  // 以 sum 返回两个复数 z1, z2 的和
```

```
// -----基本操作的实现

void add( complex z1, complex z2,
                                complex  &sum ) {
  // 以 sum 返回两个复数  z1, z2 的和
    sum.realpart = z1.realpart + z2.realpart;
    sum.imagpart = z1.imagpart + z2.imagpart;
}

{ 其它省略 }
```

1.4 算法和算法分析

一、算法

二、算法设计的原则

三、算法效率的衡量方法和准则

四、算法的存储空间需求

一、算法

算法是为了解决某类问题而规定的一个有限长的操作序列。

一个算法必须满足以下五个重要特性：

1. 有穷性　2. 确定性　3. 可行性

4. 有输入　5. 有输出

1．有穷性　对于任意一组合法输入值，在执行有穷步骤之后一定能结束，即：算法中的每个步骤都能在有限时间内完成。

2．确定性　对于每种情况下所应执行的操作，在算法中都有确切的规定，使算法的执行者或阅读者都能明确其含义及如何执行。并且在任何条件下，算法都只有一条执行路径。

3. **可行性**　算法中的所有操作都必须**足够基本**，都可以通过已经实现的基本操作运算有限次实现之。

4. **有输入**　作为算法加工对象的量值，通常体现为算法中的一组变量。有些输入量需要在算法执行过程中输入，而有的算法表面上可以没有输入，实际上已被嵌入算法之中。

5．**有输出** 它是一组与“输入”有确
定关系的量值，是算法进行信息加工后得到的结果，这种确定关系即为算法的功能。

二、算法设计的原则

设计算法时，通常应考虑达到以下目标：

1. 正确性

2. 可读性

3. 健壮性

4. 高效率与低存储量需求

1. 正确性

首先，算法应当满足以特定的“规格说明”方式给出的需求。

其次，对算法是否“正确”的理解可以有以下四个层次：

a. 程序中不含语法错误；

b. 程序对于几组输入数据能够得出满足要求的结果；

c．程序对于精心选择的、典型、苛刻且带有刁难性的几组输入数据能够得出满足要求的结果；

d．程序对于一切合法的输入数据都能得出满足要求的结果；

通常以第 c 层意义的正确性作为衡量一个算法是否合格的标准。

2. 可读性

算法主要是为了人的阅读与交流，其次才是为计算机执行，因此算法应该易于人的理解；另一方面，晦涩难读的程序易于隐藏较多错误而难以调试。

3. 健壮性

当输入的数据非法时，算法应当恰当地作出反应或进行相应处理，而不是产生莫名奇妙的输出结果。并且，处理出错的方法不应是中断程序的执行，而应是返回一个表示错误或错误性质的值，以便在更高的抽象层次上进行处理。

4. 高效率与低存储量需求

通常，效率指的是算法执行时间；存储量指的是算法执行过程中所需的最大存储空间，两者都与问题的规模有关。

三、算法效率的衡量方法和准则

通常有两种衡量算法效率的方法:

A **事后统计法**

缺点：1. 必须执行程序

2. 其它因素掩盖算法本质

B **事前分析估算法**

一个用高级语言编写的程序在计算机上运行时所消耗的时间取决于下列因素：

1. 算法选用的策略
2. 问题的规模
3. 编写程序的语言
4. 编译程序产生的机器代码的质量
5. 计算机执行指令的速度

一个特定**算法的“运行工作量”**的大小，只依赖于问题的规模（通常用整数量n表示），或者说，**它是问题规模的函数。**

假如，随着问题规模 n 的增长，**算法执行时间的增长率和 f(n) 的增长率相同**，则可记作：

$$T(n) = O(f(n))$$

称T (n) 为算法的(渐近)时间复杂度。

如何估算

算法的时间复杂度？

算法 = 控制结构 + 原操作

控制结构：顺序、分支、循环结构

原操作：固有数据类型的操作

算法的执行时间 =

$\sum$ 原操作(i)的执行次数×原操作(i)的执行时间

<u>算法的执行时间</u> 与

<u>原操作执行次数之和</u> 成正比

```
for (i=1; i<=n; ++i)                    ---------------- n
for (j=1; j<=n; ++j) {                  ---------------  n*n
   c[i,j] = 0;                          ---------------- n^2
   for (k=1; k<=n; ++k)                 -------- n^2 *n
      c[i,j] += a[i,k]*b[k,j];                --- n^3
}
```

求两个n阶矩阵的乘积

算法的频度：

$$f(n) = n + n*n + n^2 + n^2*n + n^3$$
$$= 2n^3 + 2n^2 + n$$

从算法中选取一种对于所研究的问题来说是 基本操作 的原操作，以该基本操作 在算法中重复执行的次数 作为算法运行时间的衡量准则。

(a) { ++x ; s=0 ;}

时间复杂度 $O(1)$ 常量阶

(b) for(i=1;i<=n;++i)

{ ++x ; s+=x ; }

时间复杂度 $O(n)$ 线性阶

(c) for(j=1;j<=n;++j)

for(k=1;k<=n;++k)

{ ++x ; s+=x ; }

时间复杂度 $O(n^2)$ 平方阶

四、算法的存储空间需求

算法的空间复杂度定义为:

$$S(n) = O(f(n))$$

表示随着问题规模 n 的增大，算法运行所需存储量的增长率与 f(n) 的增长率相同。

算法的存储量包括:

1. 输入数据所占空间
2. 程序本身所占空间
3. 辅助变量所占空间

若输入数据所占空间只取决于问题本身，和算法无关，则只需要分析除输入和程序之外的辅助变量所占额外空间。

若所需额外空间相对于输入数据量来说是常数，则称此算法为原地工作。

若所需存储量依赖于特定的输入，则通常按最坏情况考虑。

本章学习要点

1、熟悉各名词、术语的含义，掌握基本概念。

2、理解ADT的意义。

3、理解算法五个要素的确切含义。

4、掌握计算语句频度和估算算法时间复杂度的方法。

5、算法的表示与实现。

作业：1-1

- 1、简述下列术语：数据结构、逻辑结构、存储结构、数据抽象类型。
- 2、简述数据抽象类型的两个重要特征以及这两个重要特征的意义。
- 3、简述四种逻辑结构、两种主要的数据关系映象。
- 4、分析一下冒泡排序的时间复杂度